

POD ROYALE

Un battle royale où chaque pod est un guerrier

Les guerriers s'attaquent en appelant le Service `guerrier` : c'est le load balancing de Kubernetes qui choisit la victime au hasard. Le dernier survivant gagne.

Compte rendu de projet

INFO910 — Kubernetes

Palanca • Grandy

Octobre 2026

Table des matières

Le principe 1

1 Architecture 1

2 Notions du cours et où elles apparaissent 3

3 Installation (minikube) 3

3.1 En une commande 3

3.2 Étape par étape 4

4 Déroulé de la démo 4

5 Pourquoi pas de CORS ici 5

6 Structure du dépôt 6

Le principe

Pod Royale est un battle royale où chaque pod est un guerrier. Les guerriers s'attaquent en appelant le Service `guerrier` : c'est le load balancing de Kubernetes qui choisit la victime au hasard. Les morts échouent leur *readinessProbe* et sortent des endpoints. Le dernier survivant gagne, puis un nouveau round commence.

La figure 1 montre l'arène pendant un round : les cartes des guerriers avec leurs points de vie, le fil des combats et le tableau d'honneur.



FIGURE 1 – Aperçu de l'arène pendant un round.

1 Architecture

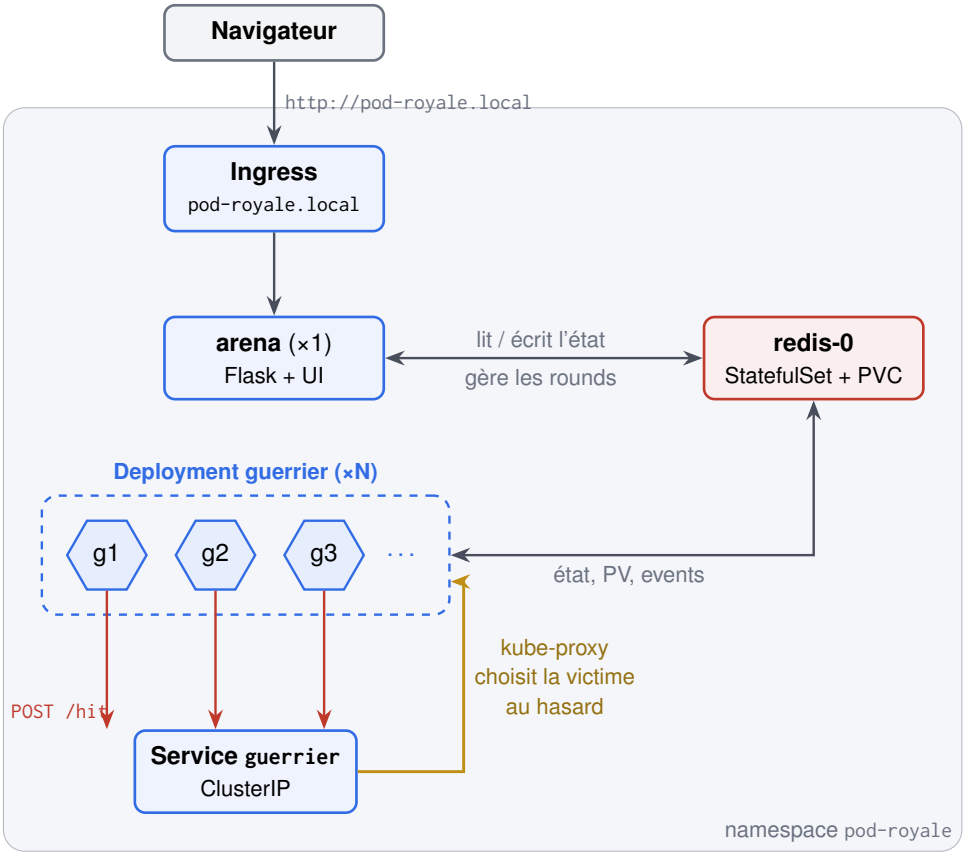


FIGURE 2 – Les composants de Pod Royale dans le cluster.

Composant	Rôle
`guerrier/`	Python 3.12 + Flask (waitress). Expose `/hit`, `/ready`, `/healthz`, `/freeze`, `/`. Une boucle d'attaque tourne dans un thread.
`arena/`	Python 3.12 + Flask (waitress). L'arbitre : il gère les rounds, sert la page (Jinja2) et `/api/state`. **1 seul réplica.**
`redis`	`redis:7-alpine` avec mot de passe et AOF, sur un PVC. Stocke l'état du jeu (clés `pr:*`). La perte de PV et la détection de mort se font dans un script Lua, donc de façon atomique.

> Chaque attaque ouvre une **nouvelle connexion TCP** (pas de `requests.Session`, en-tête `Connection: close`). Sans ça, le keep-alive collerait chaque attaquant à la même cible et le hasard du load balancing disparaîtrait.

2 Notions du cours et où elles apparaissent

Notion	Où dans le projet
Pod	Chaque guerrier est un pod. Son nom de pod donne son nom d'affichage (« Gérard le Fourbe »).
Namespace	`k8s/00-namespace.yaml` : tout vit dans `pod-royale`, et `reset.sh` supprime tout d'un coup.
Deployment	`k8s/20-guerrier-deployment.yaml` (6 réplicas, RollingUpdate `maxSurge: 2` / `maxUnavailable: 1`) et `k8s/30-arena-deployment.yaml` (1 réplica, `Recreate`).
Service et DNS	`k8s/21-guerrier-service.yaml` : la cible des attaques, le load balancing choisit la victime. `redis.pod-royale.svc.cluster.local` est résolu par le DNS du cluster.
StatefulSet	`k8s/10-redis-statefulset.yaml` : identité stable `redis-0`.
PVC	`volumeClaimTemplates` de Redis : le tableau d'honneur survit à `kubectl delete pod redis-0`.
ConfigMap (env)	`jeu-config` via `envFrom` : `ATTACK_INTERVAL_MS`, `VICTORY_PAUSE_S`, `REDIS_HOST`...
ConfigMap (volume)	La même ConfigMap, montée en `/config/classes.json` : les classes de guerriers.
Secret	`k8s/01-secret-redis.yaml` : `REDIS_PASSWORD` injecté par `secretKeyRef`. Aucun mot de passe en clair dans les Deployments.
readinessProbe	`GET /ready` renvoie 503 quand le guerrier est mort, donc il sort des endpoints en $\sim$2 s (`periodSeconds: 2`, `failureThreshold: 1`).
livenessProbe	`GET /healthz` renvoie 500 si la boucle d'attaque est figée (`/freeze`), donc le conteneur redémarre (RESTARTS +1).
startupProbe	Laisse jusqu'à 60 s au guerrier pour démarrer avant que la livenessProbe prenne le relais : sur un nœud chargé, un démarrage lent ne finit plus en CrashLoopBackOff.
requests / limits	Sur tous les conteneurs (ex. guerrier : `20m/48Mi` $\rightarrow$ `500m/128Mi`).
Downward API	`POD_NAME`, `POD_IP`, `NODE_NAME` injectés depuis `metadata.name`, `status.podIP`, `spec.nodeName`.
Arrêt propre (SIGTERM)	`terminationGracePeriodSeconds: 10` : le guerrier annonce « foudroyé par les dieux » avant de partir.
Ingress	`k8s/40-ingress.yaml` : `pod-royale.local` $\rightarrow$ `arena:80`. Seule l'arène est exposée.
Gateway API (bonus)	`k8s/bonus/41-httproute.yaml` : `Gateway` + `HTTPRoute` équivalents à l'Ingress. Non appliqués par défaut.

3 Installation (minikube)

Prérequis : Docker, `kubectl`, `minikube` (ou `kind`).

3.1 En une commande

```
./scripts/up.sh                      # macOS, Linux, WSL, Git Bash
```

```
powershell -ExecutionPolicy Bypass -File scripts\up.ps1    # Windows
```

Le script démarre Docker Desktop s'il le faut (macOS et Windows), puis minikube si aucun cluster ne répond. Il construit et charge les images, déploie et attend que tout soit prêt. Il ouvre ensuite l'arène sur **`http://localhost:8080`** par un port-forward : pas de `sudo`, pas de `minikube tunnel`, pas de fichier hosts à modifier, quel que soit l'OS.

up.sh	up.ps1	Effet
`PORT=9090`	`-Port 9090`	Change le port local.
`NO_BROWSER=1`	`-NoBrowser`	N'ouvre pas le navigateur.
`INGRESS=1`	`-Ingress`	Active l'addon ingress pour `pod-royale.local`.

Ctrl+C coupe l'accès, et le jeu continue de tourner dans le cluster.

3.2 Étape par étape

```
# 1. Cluster + contrôleur Ingress
minikube start
minikube addons enable ingress

# 2. Construire les images et les charger dans minikube
./scripts/build.sh

# 3. Déployer et attendre que tout soit prêt
./scripts/deploy.sh

# 4. Résoudre pod-royale.local vers le cluster
#    Linux, ou minikube avec un driver VM :
echo "$(minikube ip) pod-royale.local" | sudo tee -a /etc/hosts
#    macOS avec le driver docker : l'IP de minikube n'est pas
#    joignable depuis le Mac. Lancer « minikube tunnel » dans un
#    autre terminal (à laisser ouvert), puis :
echo "127.0.0.1 pod-royale.local" | sudo tee -a /etc/hosts

# 5. Ouvrir http://pod-royale.local
```

Sans Ingress :

```
kubectl -n pod-royale port-forward svc/arena 8080:80
# puis http://localhost:8080
```

Pour tout supprimer : `./scripts/reset.sh` (le PVC, et donc le tableau d'honneur, part avec le namespace).

4 Déroulé de la démo

`./scripts/demo.sh` enchaîne les étapes, avec une pause entre chacune. L'arène reste ouverte à côté.

#	Commande	Ce qu'on voit dans l'arène
1	`kubectl get all -n pod-royale`	Tous les objets du jeu.
2	`kubectl get endpoints guerrier -n pod-royale` (pendant un round)	Les morts disparaissent des endpoints (readinessProbe).
3	`kubectl scale deploy/guerrier --replicas=12`	Des renforts arrivent, en spectateurs jusqu'au prochain round.
4	`kubectl delete pod <guerrier>`	« X a été foudroyé par les dieux », puis un nouveau pod apparaît.
5	`kubectl exec <pod> -- curl -X POST localhost:8080/freeze`	Le guerrier se fige, la livenessProbe échoue, RESTARTS +1 et « X revient d'entre les morts ».
6	`kubectl delete pod redis-0`	Redis redémarre et le tableau d'honneur est conservé (PVC).
7	Patch de la ConfigMap `ATTACK_INTERVAL_MS=400` + `kubectl rollout restart deploy/guerrier`	Mode frénésie : les rounds durent quelques secondes.
8	`kubectl scale deploy/guerrier --replicas=2`	Le duel final.

Dans l'arène, chaque coup réellement échangé entre deux pods est rejoué par de petits gladiateurs : flèches pour les archers, boules de feu pour les mages, entailles à l'épée, à la hache ou à la dague, et le sysadmin qui frappe au clavier. Un **coup critique** se voit de loin : aura dorée pendant la charge, ralenti, flash et texte « CRITIQUE ! ». La **barre d'espace** met la scène en pause pour expliquer ce qui se passe.

> Pour voir le hasard du load balancing :
>
> `kubectl -n pod-royale logs -l app=guerrier --prefix | grep '→'`
>
> Chaque ligne indique quel pod a frappé quel pod.

5 Pourquoi pas de CORS ici

Le navigateur ne parle **qu'à une seule origine** : `http://pod-royale.local`, c'est-à-dire l'arène derrière l'Ingress. La page est rendue côté serveur par Jinja2, et `arena.js` appelle `fetch('/api/state')` avec une URL relative, donc sur la même origine. Il n'y a jamais de requête cross-origin, et aucun en-tête CORS n'est nécessaire.

Les guerriers et Redis ne sont **pas exposés** hors du cluster (Services ClusterIP, pas d'Ingress). L'arène lit l'état dans Redis côté serveur, puis le renvoie au navigateur. C'est plus simple et plus sûr : la surface exposée se limite à une seule application.

6 Structure du dépôt

Dossier	Contenu
`guerrier/`	`app.py`, `names.py`, `Dockerfile`
`arena/`	`app.py`, `templates/`, `static/`, `Dockerfile`
`k8s/`	Manifests numérotés dans l'ordre d'application (+ `bonus/` : Gateway API)
`scripts/`	`up.sh` / `up.ps1` (tout en un), `build.sh`, `deploy.sh`, `demo.sh`, `reset.sh`